PA 3 参考提示

数据结构课程团队

Contents

CST 3-1-2 Dance 2

CST 3-1-3 Feature 3

CST 3-2-2 Kidd 4

CST 3-3-1 Temperature 5

CST 3-3-2 Nearest Neighbor 6

通用技巧

本次 PA 的很多题目需要在平衡树中维护某些额外信息。这里展示一些常用的技巧：

- **按序查找**。我们希望查找到平衡树中第 k 大/小的项。我们可以首先对平衡树上的每个节点维护其子树的规模。在按序查询时，类似于二分查找，逐层深入，对于每一层，比较当前节点右/左子树的规模和 $k-1$ 的关系，并比较结果递归进入左或右子树，或者返回当前结点。
- **懒惰标记**。我们有时希望对某一棵子树中所有的结点执行一个操作（例如将某个维护的值 $+x$），则我们可在节点中维护一个 **“懒惰标记”**（例如当前子树所有 $+x$ 之和），当需要对某一棵子树执行操作时，我们只对**根节点**进行这一操作，同时在懒惰标记中记录此次操作。未来当需要访问到孩子节点时，先将懒惰标记下传至孩子节点，并清空当前节点懒惰标记，随后再访问孩子节点。

CST 3-1-2 Dance

思路提示

本题可以使用 Splay 来完成。其中序遍历对应着舞者在舞圈中的顺序。

对于 `I` 和 `S` 操作，只需要找到对应编号的舞者，然后进行相应的插入/交换操作。

对于 `R` 和 `F` 操作，二者都可以通过对 Splay 的分裂和合并完成。例如对于 R $r > 0$ 操作，我们只需将 Splay 分裂成 $[0, n-r)$ 以及 $[n-r, n)$ 两段，随后按照 $[n-r, n), [0, n-r)$ 的顺序进行合并。F 操作可以类似完成。

分裂和合并可以通过 splay 操作完成。若我们想将 $[0, n)$ 分裂成 $[0, k], [k+1, n)$ 两段，则可以将 k 号结点伸展到根，此时将右子树摘除即完成了分裂。合并可理解为分裂的逆操作。

翻转可以通过懒惰标记进行，避免复杂度退化。

难点提示

为了保证 Splay 的复杂度，**所有操作**都需完成 splay。伸展树单次操作均摊 $O(\log n)$ 的正确性建立在 `splay` 上，若某一操作未实现 `splay` 则无法保证时间复杂度满足要求。例如：若 `swap` 未实现 `splay`，考虑连续插入 n 个点 `I k k`, $k = 0, \ldots, n-1$，此时伸展树形态为一条链，任意 swap 操作的期望复杂度退化为 $O(n)$，未实现 `splay` 则无法均摊成 $O(\log n)$。

提示中对 R 操作的优化，一种做法是不修改子树，而是全局记录当前旋转了多少位置。对于未来的请求，反向旋转这些请求的编号来确保相对位置关系不变。由于 splay 常数较大，这种优化对于降低运行时间有一定帮助。

CST 3-1-3 Feature

思路提示

本题的难点在于修改值操作和计算加权和操作所对应的排序方式不同。

设按照权重从小到大排序的序列为 A，按权重的绝对值从小到大排序的序列为 B。此外，我们维护序列 B 的加权后缀和。三种操作分别对应：在两个序列中，先删除旧权重及其样本值，再插入修改后的权重及样本值；在序列 A 中查询第 k 大权重，并在序列 B 中修改它们的值；查询序列 B 第 k 大特征对应的加权后缀和。

这两种操作可以使用平衡树来维护。具体来说，分别对序列 A 和序列 B 维护两棵平衡树，并对序列 B 的平衡树中的每个节点，维护子树中的所有加权和。

修改权重操作需要同步更新两棵树。首先在平衡树 A 中，通过按序查找定位到权重第 k 大的特征。然后，根据该特征的权重绝对值，在 B 中找到对应节点，更新其值。查询加权和，可以直接在 B 中通过按序查找完成。

难点提示

两棵平衡树仅存储的内容及更新方式不同，可以通过对 BBST 进行合理封装避免重复代码。

CST 3-2-2　Kidd

思路提示

本题可以使用线段树（07-B 节）解决。我们可以在每个节点记录其对应区间的翻转操作总和。

对于区间修改，我们可以采用懒惰标记保证单次操作的复杂度。

在处理大值域时，类似于讲义中的做法，可采用离散化。首先将所有区间端点排序并去重。这些端点将数轴分割成若干个初级区间。从而只需对这些区间进行建树。

难点提示

可以通过数组紧凑表示线段树，以节省空间，此时需要注意数组大小和叶子节点数的关系。

注意计算中间结果时是否需要使用 `long long`，尤其注意若两个变量均为 `int`（未进行类型转换），则进行加法或乘法计算的结果也为 `int`。

CST 3-3-1　Temperature

思路提示

方法一：K-D 树（07-D 节）

我们可以构建一棵 K-D 树来组织平面上的所有点。建树时，交替选择 x 轴和 y 轴作为分割维度，以点的坐标中位数进行划分，递归构建子树。每个节点需要额外存储其所代表子空间内所有点的温度最大值和最小值。

查询时，利用这些预存的最值进行剪枝。如果一个节点代表的区域与查询矩形完全不相交，则直接跳过。如果完全包含，则用该节点的最值更新答案。若部分相交，则递归查询子节点。如果一个节点区域内的最大值已小于当前找到的最大值（或最小值已大于当前找到的最小值），则该区域内不可能有更优解，可以直接剪枝。为实现该剪枝，查询最大值和最小值可分开进行。

方法二：双层搜索树（“树套树”, 07-C3 节）

另一种方法是使用树套树，例如外层用平衡树或线段树按 x 坐标组织，内层再用一个数据结构按 y 坐标组织。

具体实现可以是，外层构建一棵基于 x 坐标的线段树。线段树的每个节点内，再构建一棵基于 y 坐标的平衡树（线段树），用于存储对应 x 区间内所有点。同样，每个内层树的节点也需维护其子树内的温度最值信息，以便进行高效查询和剪枝。

难点提示

为避免超时，建议按照如上介绍的方式进行剪枝。此外，也可以尝试在叶子节点多存储几条记录，从而利用内存连续性。

如使用 QuickSelect（14-B3 节），可以检查其在处理大量相同值时是否会退化。

你可以在本地构建一组测例，并通过比较优化前后的运行时间来评估优化的有效性（部分剪枝与数据特征相关，在构造时可以留意）。

CST 3-3-2　Nearest Neighbor

思路提示

本题可以使用 K-D 树解决。具体来说，使用 K-D 树来存储所有向量。建树时，交替选择不同维度作为分割轴，通过点的坐标中位数将空间划分为两个子空间，并递归地对子空间进行建树。

查询操作可从 K-D 树的根节点开始遍历。朴素遍历复杂度为线性，因此需要进行一定的剪枝。可以优先访问查询点所在的子树或距离更近（可以采用近似方式，例如只考虑当前维度）的子树，这些子树更有可能产生最近点。在回溯时，若当前的最近距离已经小于待查点到另一子树区域的距离（是否可以近似计算?），则可进行剪枝。

由于本题的数据为随机生成，因此此类剪枝通常能够产生较为良好的效果。（思考：若数据并非均匀随机生成，单次查询最坏复杂度是否可能达到 $\Omega(n)$?）

难点提示

可以在叶子节点上保存多于一个点，当查询走到叶子时，暴力遍历叶子上的所有的点。这样做的好处是可以减少树的层数，减少递归深度，并利用连续访问的性质优化缓存命中。

剪枝的计算开销（例如，是计算点到区域的精确值还是估一个下界）和剪枝的有效性需要进行权衡。你可以在本地构建一组测例，并通过比较优化前后的运行时间来评估优化的有效性。

拓展

对于许多现实应用，我们并不需要找到 “最近” 的向量，而只需要找到 “比较近” 的若干个向量。如何在这种查询场景下实现比 K-D 树更高的计算效率是当前的研究热点。感兴趣的同学可以以 “High-dimensional Vector Search” “Approximate Nearest Neighbor Search” 等为关键词进行进一步探索。